

Diagnose R9700 Mapping Issues

08/23/2026 16:19:22

<https://chatgpt.com/c/6a8b1b3a-b178-83ea-b486-083c67d02ab0>

I'll trace the new work to the first divergence from TinyGPU/mac-amdgp...



I'll trace the new work to the first divergence from TinyGPU/mac-amdgpu, validate likely causes against driver code and hardware semantics, then prioritize concrete debugging experiments.

The new A/B evidence shifts the blocker from BAR/mapping to kernel-specific RMSNorm inverse-root code generation. I'm validating that against RS64 semantics and avoiding earlier MQD/disassembly overinterpretations.

Mac-amdgpu v0.1.48 adds relevant Apple/TB5 caveats— inert BAR2 doorbells, SDMA MMIO-WPTR fallback, and zeroed DART-backed GART reads—so I'll audit address domains and completion separately.

I'm splitting the NaN-producing original image from newly generated images that stall, and testing instruction-cache coherency at reused VAs. I'll also compare an independent gfx1201 ROCr-lite queue implementation.

Assessment

I reviewed the second transcript. You have made substantial progress, but the debugging has begun to conflate **three different failure domains**:

1. **Cold-start and queue-lifecycle state**: after a power cycle, the device sometimes needs a fuller GC/MEC initialization or queue reset before native compute is reliable.
2. **Program-image launch reliability**: the original RMSNorm image executes, while some newly generated images do not retire.
3. **RMSNorm numerical correctness**: the original image completes but returns NaNs.

The strongest conclusion is that you are **not currently blocked by a generic PCIe BAR, GART, or data-buffer mapping failure**. The byte-exact `hidden` result proves that your native path can map code, model data, output data, a compute ring, and completion machinery well enough to execute a real kernel correctly. The transcript reports 2,048 finite values and an exact oracle SHA-256 for that stage. `native-r9700-session-full-2.jsonl`

The original RMSNorm image then completes through the same resident machinery but produces non-finite output rather than timing out. `native-r9700-session-full-`

2.jsonl

By contrast, the regenerated rsqrt image reaches a resident-dispatch timeout with the doorbell hit, HQD active, queue RPTR at `0x31`, WPTR at `59`, and the timeline still zero.

`native-r9700-session-full-2.jsonl` The epsilon diagnostic image later exhibits a similar timeout signature. `native-r9700-session-full-2.jsonl`

Those are not all manifestations of one mapping bug.

The most important overlooked issue: newly uploaded code may not be instruction-cache coherent

This is the first thing I would test.

Your experiments repeatedly regenerate code images and load them into the resident-code path while keeping the GPU and VMID alive. CPU or BAR0 readback proves that the expected bytes reached the physical VRAM aperture. It does **not** prove that a subsequent GPU instruction fetch through GFXHUB sees those same bytes rather than cached instructions associated with the previous program at that GPU virtual address.

The tinygrad steady-state dispatch path calls:

```
self.acquire_mem(gli=0, gl2=0)
```

before programming `COMPUTE_PGM_LO`, `COMPUTE_PGM_RSRC1/2/3`, scratch state, user data, and `DISPATCH_DIRECT`.

That is reasonable as a normal dispatch barrier when programs have already been installed and their lifetimes are managed by tinygrad. It is not sufficient evidence for the separate operation of **overwriting executable memory and immediately executing the replacement image**. The same helper is capable of emitting GLI, GLK, GL1, GL2, GLM, and GLV invalidation/writeback controls.

Add an explicit code-install barrier

For the first diagnostic build, make the upload sequence:

1. Prove the queue is idle.
2. Copy the new code image.
3. Read it back on the CPU as you already do.
4. Flush the host/HDP path.

5. Submit an `ACQUIRE_MEM` on the compute queue with a deliberately broad cache operation:

- `GLI_INV`
- `GLK_INV` and `GLK_WB`
- `GL1_INV`
- `GL2_INV` and `GL2_WB`
- `GLM_INV` and `GLM_WB`
- `GLV_INV`

6. Only then program `COMPUTE_PGM_*` and dispatch.

For diagnosis, using address zero and the full supported range, as tinygrad's helper permits, is preferable to accidentally getting the range encoding wrong. Once the behavior is proven, this can become a code-range-specific, code-upload-only operation rather than an expensive barrier before every dispatch.

Also run the same replacement image at a **fresh GPU VA backed by fresh physical VRAM pages**, without reusing the old code slot. The four possible outcomes are especially informative:

Result	Meaning
Cache invalidation fixes it	Stale instruction/cache state was the blocker
Fresh VA fixes it, invalidation does not	Cache tagging or invalidation-range encoding is wrong
Neither fixes it	Descriptor, program contents, queue state, or actual shader behavior
Only a physical power cycle fixes it	Reset/firmware/cache lifecycle remains incomplete

I would additionally map at least one valid read/execute padding page after each program image during this test. That rules out instruction prefetch reaching an unmapped page at the end of a short replacement image.

A second major issue: the failing tests may not be independent

A timed-out or faulting compute queue should be considered poisoned until you have explicitly removed and reset it.

The transcript first runs the rsqrt image, which times out, and later obtains a similar result from the epsilon image. Unless a complete HQD/MEC reset occurred between those tests-and the individual failure records do not establish that-the epsilon result is not independent evidence that the epsilon kernel itself hangs. It may simply have been submitted into residual state left by the first failure.

Use a “sentinel sandwich” for every experimental image:

1. Start from a reset queue.
2. Run the known-good add-one or embed kernel.
3. Confirm the expected fence and byte-exact output.
4. Run exactly one candidate image.
5. Salvage all output and fault state even if it times out.
6. Try the known-good sentinel again.
7. Tear down and reset the queue before testing the next image.

A failed post-candidate sentinel tells you that the candidate wedged the queue. It does not tell you that the next candidate is broken.

Recent Linux GFX12/MES code uses a much stronger teardown than merely clearing RPTR/WPTR: it writes `CP_HQD_DEQUEUE_REQUEST = 0x2`, asserts `SPI_COMPUTE_QUEUE_RESET`, and waits for `CP_HQD_ACTIVE` to clear. Recent upstream work also corrected RS64 reset-path selection and MEC pipe-reset ordering so that active HQDs are cleared while the appropriate reset remains asserted.

Your native teardown should therefore verify, in order:

- the correct RS64 versus legacy MEC reset path;
- dequeue request issued;
- compute queue reset asserted;
- `CP_HQD_ACTIVE == 0`;

- full 64-bit RPTR/WPTR carrier memory reset, not just the low register;
- stale exception/fault observations recorded before reactivation;
- pipe reset released only after every selected HQD is inactive.

Upstream MES work has specifically addressed stale high halves of 64-bit queue pointers, so this is worth making explicit even though the displayed `hqd_pq_wptr_hi` happened to be zero in the captured timeout.

Two current interpretations in the transcript need correction

1. `CP_MEC_RS64_EXCEPTION_STATUS` is not the shader program counter

The GFX12 register definition decodes the low bits as:

- bit 0: illegal instruction
- bit 1: misaligned address
- bit 2: unaligned instruction
- bit 3: page fault
- bits 26:4: RS64 exception instruction address.

Therefore `0x0000c67a` has:

```
misaligned-address bit = 1
page-fault bit        = 1
RS64 instruction field = 0x0c67
```

But this is the **MEC RS64 command-processor firmware exception state**. It is not an SQ wave PC and should not be converted into an offset in the RMSNorm shader's `.text` section. Linux's GFX12 diagnostic paths treat the MEC RS64 pointer and SQ shader wave PCs as distinct state.

It could indicate the MEC faulting while processing an address-bearing PM4 packet, such as:

- an indirect buffer;
- a queue pointer;

- a `WAIT_REG_MEM` address;
- a completion or `RELEASE_MEM` address;
- another CP-side memory operation.

It does not by itself prove that the shader instruction fetch or a shader load/store faulted.

Furthermore, the bitfield may be sticky. Record it **before and after each candidate dispatch**. A value that was already present before the dispatch is not evidence about that dispatch.

The corresponding shader-side evidence should come from:

- `GCVM_L2_PROTECTION_FAULT_ADDR_L032/HI32` ;
- `GCVM_L2_PROTECTION_FAULT_STATUS_L032/HI32` , including the client ID;
- `CP_HQD_ERROR` ;
- CPC/CPF UTCL1 status;
- SQ wave status, trap status, and wave PC where accessible.

Linux's GFX12 register dump explicitly includes both GCVM protection-fault state and separate CP/MEC state, which is the diagnostic separation you want.

2. `COMPUTE_PGM_RSRC3` is not encoding VGPR/SGPR usage here

On GFX12, the relevant `COMPUTE_PGM_RSRC3` fields are:

- bits 3:0: shared-VGPR count;
- bits 11:4: `INST_PREF_SIZE` ;
- bit 13: GLG enable;
- bit 31: image-op enable.

For these values, the changing field is instruction prefetch size, not normal VGPR or SGPR allocation. With the documented 128-byte granularity:

- `0x40` means a field value of 4, or 512 bytes;
- `0x80` means 8, or 1,024 bytes;

- `0xa0` means 10, or 1,280 bytes.

That makes the earlier resource-geometry theory based on `32 → 64 → 128 → 160` unreliable.

It also suggests a very clean experiment:

Known-good original RMSNorm:

```
generated RSRC3 = 0xa0
forced RSRC3    = 0x00
forced RSRC3    = 0x80
```

Rsqrt replacement:

```
generated RSRC3 = 0x80
forced RSRC3    = 0x00
```

Epsilon probe:

```
generated RSRC3 = 0x40
forced RSRC3    = 0x00
```

Do this without modifying the images or their hashes; temporarily override only the programmed register in a diagnostic build.

Interpretation:

- If a replacement runs with `RSRC3=0`, investigate instruction prefetch across an invalid page or incoherent instruction-cache state.
- If the old image fails only after changing `RSRC3`, the prefetch field itself is materially involved.
- If every image behaves the same regardless of `RSRC3`, stop treating that register as causal.

The old image's larger prefetch value already works, so a simple "prefetch size too large" theory is not a good fit. A prefetch/cache interaction involving **which bytes were newly installed at which VA**, however, remains plausible.

The static disassembly does not yet prove an LLVM division bug

The decoded sequence involving:

- square root;
- `V_DIV_SCALE_F32 ;`
- `V_RCP_F32 ;`
- Newton-style FMAs;
- `V_DIV_FMAS_F32 ;`
- `V_DIV_FIXUP_F32 ;`

is a normal form of LLVM's precise floating-point division lowering on AMD GPUs. Its presence is not proof that the compiler applied an incorrect `2^32` scaling factor. Comparable LLVM reports and backend output show this family of sequences as the canonical full-precision division implementation.

There are two weaknesses in the current conclusion:

1. The hand decode did not establish a complete register/dataflow graph through the whole sequence.
2. Splitting a C expression into local variables does not force LLVM to preserve those boundaries; optimization can recombine it into the original expression.

There is also now the cache-coherency concern: a newly built “split” image that appears to behave like the old image may literally be executing old cached instructions.

The compiler hypothesis remains credible, but it should be tested after launch and code visibility are made trustworthy.

Make the timeout tell you where execution stopped

At present, “timeline remains zero” is being interpreted as “the kernel did not complete.” That may be true, but it is not the only explanation. The kernel may have:

- partially or fully executed;
- written its output;
- stalled at `CS_PARTIAL_FLUSH ;`
- failed during `RELEASE_MEM ;`

- written the fence to GPU memory while the CPU-visible mapping remained stale.

Add three or four diagnostic markers to the PM4 stream:

```
A: non-EOP CP memory write before DISPATCH_DIRECT
B: non-EOP CP memory write immediately after DISPATCH_DIRECT
C: memory write after CS_PARTIAL_FLUSH
D: normal completion/timeline RELEASE_MEM
```

Use a BAR0-visible VRAM diagnostic page initially, so the marker itself does not depend on the DriverKit/DART system-memory path.

The outcomes distinguish the stages:

Markers observed	Interpretation
None	Ring fetch or packet parsing failed before the dispatch
A only	Dispatch packet or associated register state caused CP failure
A and B, not C	Dispatch was accepted, but compute did not drain through partial flush
A, B, C, not D	Completion packet/address or system-memory fence path
All markers, CPU timeline zero	CPU/system-memory visibility problem
Output changed but C absent	Shader ran at least partially before the stall

Also preserve and inspect the candidate output after every timeout. Initialize it with a recognizable pattern and report:

- number of changed dwords;
- first and last changed offsets;
- raw first 64 output bytes;
- whether the expected constant appeared anywhere.

The epsilon diagnostic currently throws away its most useful evidence when the fence times out.

Annotate all 59 PM4 dwords

The queue state of WPTR 59 and HQD RPTR 49 is valuable, but only after you print the actual packet boundaries.

Dump something like:

```
ring[00..07] ACQUIRE_MEM
ring[08..??] SET_SH_REG: COMPUTE_PGM_LO/HI
ring[??..??] SET_SH_REG: RSRC1/2/3
ring[??..??] SET_SH_REG: USER_DATA
ring[??..??] DISPATCH_DIRECT
ring[??..??] EVENT_WRITE: CS_PARTIAL_FLUSH
ring[??..58] RELEASE_MEM
```

Normalize the hardware RPTR by the ring size and identify whether dword 49 is:

- a packet header;
- a packet payload;
- the first dword following a completed packet.

Until that mapping exists, it is premature to say that the MEC fetched or executed the dispatch packet. It is only safe to say that the queue pointer advanced.

Verify code through the GPU's address translation, not only through BAR0

For every experimental image:

1. Record the CPU-side SHA-256.
2. Record the physical/BAR0 readback SHA-256.
3. Use SDMA or a known-working copy kernel to copy the program and descriptor **from their GPU virtual addresses** to a BAR0-visible scratch page.
4. Hash the copied bytes.
5. Record the full PTE walk for:

- descriptor VA;
- program-entry VA;
- last byte of `.text` ;
- final mapped image byte;
- instruction-prefetch padding page.

This separates:

```
CPU upload correct
  from
VRAM physical storage correct
  from
GPUVM translation correct
  from
instruction-cache contents correct
```

A BAR0 readback can establish only the first two.

Given the RS64 page-fault bit, also correlate the GFXHUB fault address with every address-bearing PM4 field. If the reported address equals the timeline, kernarg, ring, code entry, or a page immediately after `.text` , the problem becomes much more mechanical.

Compare the entire kernel descriptor, not just the digest and RSRC words

For the old RMSNorm, rsqrt RMSNorm, epsilon probe, and zero-store control, emit one structured descriptor report containing:

```
image SHA-256
image size
descriptor file offset
entry file offset
resident GPU VA
entry GPU VA
entry alignment

RSRC1
RSRC2
```

RSRC3

wave32/wave64 property
actual DISPATCH_INITIATOR.CS_W32_EN
user SGPR count and enable bits
kernarg segment size and alignment
private segment size
group segment size
dispatch-pointer enable
kernarg-pointer enable
scratch enable and required scratch size
code object version
AMDGPU target ID and feature string
.text offset, size, and page coverage

This is important because tinygrad explicitly derives the wave32 dispatch bit from the program's properties and programs scratch base and temporary-ring state before dispatch. A newly generated image whose descriptor requests a different wave mode, scratch configuration, user-SGPR layout, or dispatch-pointer preload can fail even when `RSRC1/2/3` appear superficially similar.

The specific checks I would prioritize are:

1. Does `CS_W32_EN` match the new descriptor?
2. Is `private_segment_size` still zero?
3. Are the enabled user SGPRs identical?
4. Does the kernel expect a dispatch pointer that the resident path does not provide?
5. Is the program entry still 256-byte aligned after adding the resident base?
6. Are all pages touched by instruction prefetch mapped executable/readable?

Only after launch is deterministic: isolate the NaN in one-operation kernels

Do not continue changing the full RMSNorm kernel until a replacement image can be installed and executed reliably.

Then use genuinely minimal kernels: one workgroup, one active workitem, one or a few 32-bit stores, no 2,048-element output loop.

A useful sequence is:

```
K0: store raw epsilon bits
K1: store raw first input and scale values
K2: store sum_of_squares
K3: store mean_square
K4: store mean_square + epsilon
K5: store sqrt(mean_square + epsilon)
K6: store reciprocal of K5
K7: store rsqrt(mean_square + epsilon)
K8: store final normalized value before fp16 conversion
K9: store final fp16 bits
```

For `epsilon = 1e-5f`, K0 should store:

```
0x3727c5ac
```

For zero input:

```
mean_square           = 0.0f
mean_square + epsilon ≈ 9.999999747e-6
sqrt                  ≈ 0.0031622776
inverse sqrt          ≈ 316.22777
expected fp16 inverse value = 0x5cf1
```

Make the compiler boundaries real by using one kernel per operation, or storing and reloading through volatile memory. Merely introducing local C variables is not enough.

Build each microkernel under:

- optimization disabled;
- normal optimization;
- contraction disabled;
- the exact production COMGR flags.

Then use an official disassembler rather than the partial custom decode. AMD's GPUOpen ISA material now includes RDNA-family machine-readable specifications and an ISA decoder suitable for this work.

If K5 is finite but K6 is NaN, the division lowering is implicated. If K7 itself hangs only after cache/reset isolation, then the `V_RSQ_F32` path or its generated descriptor becomes a concrete target. If K0 is already wrong, the compiler is innocent and the kernarg ABI is the problem.

How I would rank the current hypotheses

Replacement-image timeout

1. **Queue state was not completely reset after a prior fault** - high confidence that this can contaminate later tests; unknown whether it explains the first rsqrt timeout.
2. **Executable-code cache coherency when replacing an image at a reused VA** - medium-high probability and currently untested.
3. **Descriptor or dispatch-property mismatch**, especially wave32, scratch, or user-SGPR state - medium probability.
4. **Instruction-prefetch interaction with page coverage or stale code**, represented by `RSRC3.INST_PREF_SIZE` - medium probability and easy to test.
5. **GPUVM mapping of the particular replacement program pages** - possible, but localized rather than a general GART failure.
6. **The rsqrt instruction itself wedges gfx1201** - possible, but not established by the MEC RS64 register.
7. **Global MQD endian/address bug** - low probability; your later sentinel calibration and successful kernels contradict it.

The tinygrad ring encoding you re-derived is indeed based on `ring_address >> 8` for `CP_HQD_PQ_BASE`, while RPTR/WPTR report addresses are stored as ordinary split 64-bit addresses. The earlier byte-swap theory should remain retired unless a new byte-level experiment contradicts that.

Original RMSNorm NaN

1. **Bad arithmetic input or intermediate**, including an incorrectly loaded epsilon - still not ruled out directly.
2. **Compiler/backend lowering or ISA behavior in the precise sqrt/division sequence** - plausible, but the static sequence alone is not proof.

3. **Cached old code invalidating the source-level decomposition experiments** - plausible until explicit GLI invalidation or fresh VA testing.
4. **FP mode, wave mode, or descriptor mismatch** - lower probability because the image completes, but worth logging.
5. **Generic input/output mapping failure** - low probability because the hidden stage is exact and the RMSNorm dispatch completes.

What to borrow from mac-amdgpu-and what not to borrow

The most useful mac-amdgpu findings are its concrete GFX12.1/NBIF register corrections, doorbell fallback, and explicit handling of DriverKit/DART-backed memory limitations. Its v0.1.48 change reports that, on its Apple Silicon/TB5 configuration, BAR2 doorbells were inert, SDMA needed an MMIO-WPTR fallback, and GPU reads through its DART-backed GART returned zero.

For your present failure:

- `doorbell_hit=1` and an advancing HQD RPTR mean the BAR2 doorbell is not the leading suspect for that run.
- The byte-exact hidden kernel means mac-amdgpu's "sysmem GPU reads return zero" observation is not a universal property of macOS or the R9700. It reflects that project's current mapping implementation and address domain.
- Its queue teardown and MMIO-WPTR techniques are useful references.
- Its fail-closed behavior around unsupported system-memory reads is worth copying.
- Its limitations should not replace the direct evidence from your TinyGPU path.

A recent independent gfx1201 macOS/ROCr-lite effort also reports that direct-MEC queues require careful packet-wrap handling, explicit WPTR/HDP/doorbell sequencing, robust dequeue plus SPI queue reset, and eventually MES-based queue management for longer-lived workloads. That is valuable first-hand corroboration, though it is community engineering evidence rather than an authoritative hardware specification.

The next concrete implementation order

I would stop modifying RMSNorm math and make these changes first:

1. **Implement deterministic queue teardown and fresh-start assertions.**
2. **Add a code-upload-specific full cache invalidation including GLI.**
3. **Add fresh-VA/fresh-page code-image allocation as a diagnostic option.**
4. **Add `--override-rsrc3` and test `0x00`, generated value, and old value.**
5. **Add PM4 word-by-word packet annotation and A/B/C/D markers.**
6. **Salvage output and fault registers after every timeout.**
7. **Verify the code image through a GPUVM-mediated copy.**
8. **Compare the complete old/new descriptors, especially wave32 and scratch.**
9. **Only then run one-operation sqrt, reciprocal, and rsqrt microkernels.**

The likely immediate resolution is not another change to the RMSNorm expression. It is making program installation and post-fault queue recovery deterministic enough that a newly generated image is guaranteed to be the image the GPU actually executes. Once that is established, the NaN should become a much smaller arithmetic or ABI problem rather than an ambiguous mixture of compiler, VM, cache, and MEC state.

The next most useful artifact for line-by-line review is the current diff containing the code upload/cache barrier, `build_compute_dispatch_words` or equivalent PM4 builder, resident descriptor decoding, and the HQD/MEC teardown path.